4D

10 November 2026

Contents

run all files in order

1 cron update in linux 3

2 scrap.py 4

3 scrape_dictionary.py 8

4 api.py 11

5 update_scheduler.py 13

6 locai_ai_analyzer.py 16

```
crontab -e
0 18 * * * /home/jerry/PycharmProjects/WelcomeScreen/.venv/bin/python
/home/jerry/PycharmProjects/WelcomeScreen/update_scheduler.py >>
/home/jerry/PycharmProjects/WelcomeScreen/cron_log.log 2>&1
#min 24basedhour day mth day-of-week

cat/home/jerry/PycharmProjects/WelcomeScreen/cron_log.log
```

```python
from datetime import datetime, timedelta
import json
import os
import re
import sqlite3
import time
from bs4 import BeautifulSoup
import requests

# ==========================================
# SETUP & ABSOLUTE PATH CONFIGURATION
# ==========================================
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUTPUT_FILE = os.path.join(BASE_DIR, "west_malaysia_10years_4d.json")
DB_FILE = os.path.join(BASE_DIR, "lotto.db")

print(f"📁 Target Output File Path: {OUTPUT_FILE}")
print(f"🗄️ Target Database Path: {DB_FILE}")


def init_db():
  """Initializes the SQLite database and creates the draws table if missing."""
  conn = sqlite3.connect(DB_FILE)
  cursor = conn.cursor()
  cursor.execute("""
        CREATE TABLE IF NOT EXISTS draws (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            company TEXT NOT NULL,
            draw_date TEXT NOT NULL,
            first_prize TEXT,
            second_prize TEXT,
            third_prize TEXT,
            special_prizes TEXT,      -- Stored as a JSON string array
            consolation_prizes TEXT,  -- Stored as a JSON string array
            UNIQUE(company, draw_date)
        )
    """)
  conn.commit()
  conn.close()


def get_already_processed_dates():
  """Reads the SQLite database to check for existing entries and avoid duplicates."""
  init_db()
  processed = set()
  try:
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("SELECT draw_date, company FROM draws")
    rows = cursor.fetchall()
    for row in rows:
      processed.add(f"{row[0]}_{row[1]}")
    conn.close()
    print(
        f"🔄 Resuming... Found {len(processed)} existing entries in the database."
    )
  except Exception as e:
```

```
    print(f"⚠️ Error reading database cache: {e}")
  return processed


def save_to_database(record):
  """Saves a single record directly to the SQLite database safely."""
  try:
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

    company = record["company"]
    date = record["date"]
    first = record["prizes"]["first"]
    second = record["prizes"]["second"]
    third = record["prizes"]["third"]
    special = json.dumps(record["prizes"]["special"])
    consolation = json.dumps(record["prizes"]["consolation"])

    cursor.execute(
        """
            INSERT OR IGNORE INTO draws
            (company, draw_date, first_prize, second_prize, third_prize, special_prizes,
consolation_prizes)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        """,
        (company, date, first, second, third, special, consolation),
    )
    conn.commit()
    conn.close()
  except Exception as e:
    print(f"❌ [DB Error] {e}")


def save_locally_safe(record):
  """Safe sequential write function for both JSON backup and SQLite database."""
  try:
    # 1. Save to JSON backup file
    if os.path.exists(OUTPUT_FILE):
      with open(OUTPUT_FILE, "r", encoding="utf-8") as f:
        try:
          data = json.load(f)
        except json.JSONDecodeError:
          data = []
    else:
      data = []

    data.append(record)

    with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
      json.dump(data, f, indent=4, ensure_ascii=False)

    # 2. Save directly to SQLite Database
    save_to_database(record)

    print(
        f"✅ [Saved] {record['company']} | Date: {record['date']} | 1st:"
        f" {record['prizes']['first']} | Special: "
```

```
        f"{len(record['prizes']['special'])} | Consolation:"
        f" {len(record['prizes']['consolation'])}"
    )
  except Exception as e:
    print(f"❌ [Save Error] {e}")


def crawl_4dmoon_date(current_date, processed_set):
  """Crawls past draw results sequentially with built-in delays for complete data."""
  date_str = current_date.strftime("%Y-%m-%d")
  headers = {
      "User-Agent": (
          "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,"
          " like Gecko) Chrome/122.0.0.0 Safari/537.36"
      ),
      "Accept-Language": "en-US,en;q=0.9",
  }

  url = f"https://www.4dmoon.com/past-results/{date_str}"

  try:
    # Polite delay to prevent server throttling or packet drops
    time.sleep(0.3)

    response = requests.get(url, headers=headers, timeout=15)
    if response.status_code == 200:
      soup = BeautifulSoup(response.text, "html.parser")
      containers = soup.find_all(["div", "section", "table"])

      for container in containers:
        text_content = container.get_text()

        target_operators = ["Magnum 4D", "Damacai", "SportsToto"]
        matched_op = next(
            (op for op in target_operators if op in text_content), None
        )

        if matched_op:
          company_name = (
              "Damacai"
              if "Damacai" in matched_op
              else ("Magnum 4D" if "Magnum" in matched_op else "Sports Toto")
          )

          # Check if this specific company and date were already logged
          record_key = f"{date_str}_{company_name}"
          if record_key in processed_set:
            continue

          all_cells = container.find_all(["td", "div", "span", "p"])
          numbers = []
          for cell in all_cells:
            val = cell.get_text().strip()
            if re.fullmatch(r"\d{4}", val):
              if val not in numbers:
                numbers.append(val)

```

```
171           if len(numbers) >= 3:
172             special_prizes = numbers[3:13] if len(numbers) >= 13 else numbers[3:]
173             consolation_prizes = numbers[13:23] if len(numbers) >= 23 else []
174 
175             # Safety check: if extraction missed numbers due to layout glitches, skip
empty
176             if len(special_prizes) == 0 or len(consolation_prizes) == 0:
177               continue
178 
179             record = {
180                 "company": company_name,
181                 "date": date_str,
182                 "prizes": {
183                     "first": numbers[0],
184                     "second": numbers[1],
185                     "third": numbers[2],
186                     "special": special_prizes,
187                     "consolation": consolation_prizes,
188                 },
189             }
190             save_locally_safe(record)
191             processed_set.add(record_key)
192   except Exception as e:
193     print(f"⚠️ Error fetching {date_str}: {e}")
194 
195 
196 def run_crawler():
197   start_dt = datetime(2016, 1, 1)
198   end_dt = datetime.now()
199 
200   processed_set = get_already_processed_dates()
201 
202   date_list = []
203   curr = start_dt
204   while curr <= end_dt:
205     if curr.weekday() in [2, 5, 6]:  # Wed, Sat, Sun
206       date_list.append(curr)
207     curr += timedelta(days=1)
208 
209   print(
210       f"🚀 Starting reliable sequential crawler for {len(date_list)} total draw"
211       " dates..."
212   )
213 
214   completed = 0
215   for d in date_list:
216     completed += 1
217     if completed % 50 == 0:
218       print(f"📊 Progress: Checked {completed}/{len(date_list)} draw dates...")
219     crawl_4dmoon_date(d, processed_set)
220 
221 
222 if __name__ == "__main__":
223   start_time = time.time()
224   run_crawler()
225   print(f"✨ Crawling finished! Data saved to SQLite DB: {DB_FILE}")
226   print(f"⏱️ Total time taken: {time.time() - start_time:.2f} seconds.")
```

```python
import json
import os
import re
import sqlite3
import time
from bs4 import BeautifulSoup
import requests

# ==========================================
# SETUP & CONFIGURATION
# ==========================================
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
JSON_OUTPUT = os.path.join(BASE_DIR, "4d_dictionary.json")
DB_FILE = os.path.join(BASE_DIR, "lotto.db")

print(f"📁 Target Dictionary JSON: {JSON_OUTPUT}")
print(f"🗄️ Target Database: {DB_FILE}")


def init_dictionary_db():
  """Initializes the SQLite database table for the 4D dictionary."""
  conn = sqlite3.connect(DB_FILE)
  cursor = conn.cursor()
  cursor.execute("""
        CREATE TABLE IF NOT EXISTS dictionary (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            term TEXT NOT NULL,
            number TEXT NOT NULL,
            UNIQUE(term, number)
        )
    """)
  conn.commit()
  conn.close()


def save_dictionary_record(term, number):
  """Saves a single dictionary term and its 4D number to SQLite and JSON."""
  try:
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute(
        """
            INSERT OR IGNORE INTO dictionary (term, number)
            VALUES (?, ?)
        """,
        (term, number),
    )
    conn.commit()
    conn.close()
    return cursor.rowcount > 0
  except Exception as e:
    print(f"❌ [DB Error] {e}")
    return False


def scrape_all_via_load_more():
  init_dictionary_db()
```

```python

url = "https://4d.today/wp-admin/admin-ajax.php"
headers = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,"
        " like Gecko) Chrome/122.0.0.0 Safari/537.36"
    ),
    "X-Requested-With": "XMLHttpRequest",
}

offset = 0
step = 20  # Usually loads batches of 20 or 60 per request
total_new_saved = 0
seen_entries = set()

print("🚀 Starting complete dictionary crawl using 'Load More' offsets...")

while True:
  payload = {
      "action": "load_more_dictionary_entries_4ddictionary",
      "offset": offset,
  }

  try:
    time.sleep(0.25)  # Polite delay to avoid hammering the server
    response = requests.post(url, data=payload, headers=headers, timeout=20)

    if response.status_code != 200:
      print(f"⚠️ Received status code {response.status_code} at offset {offset}")
      break

    resp_text = response.text.strip()
    # If the server returns empty text or whitespace, we've hit the end of the database
    if not resp_text or resp_text == "0":
      print(f"✨ Reached the end of dictionary records at offset {offset}.")
      break

    soup = BeautifulSoup(resp_text, "html.parser")
    cards = soup.find_all(
        ["div", "article", "li", "section"], class_=True
    ) or soup.find_all("div")

    if not cards or len(cards) == 0:
      print(f"✨ No more cards returned at offset {offset}. Crawl complete.")
      break

    found_on_this_page = 0
    for card in cards:
      text = card.get_text(separator=" | ", strip=True)
      match = re.search(r"\b(\d{3,4})\b", text)
      if match:
        number = match.group(1)
        term = text.replace(number, "").strip(" |.-")
        term = re.sub(r"\s+", " ", term)

        if (
```

```
                len(term) > 2
                and "Copyright" not in term
                and "4D Today" not in term
            ):
              identifier = (term, number)
              if identifier not in seen_entries:
                seen_entries.add(identifier)
                if save_dictionary_record(term, number):
                  total_new_saved += 1
                  found_on_this_page += 1

        print(
            f"📦 Offset {offset}: Processed batch. Newly saved so far:"
            f" {total_new_saved}"
        )

        # If a batch returns items, increment offset. Let's dynamically detect step size or
default to 20/60
        # If cards returned fewer than expected or 0 new items multiple times, it will
safely terminate next loop.
        offset += step

        # Safety guard to prevent infinite loops (adjust upwards if dictionary has more
than 15,000 items)
        if offset > 15000:
          print("⚠️ Reached safety offset limit.")
          break

      except Exception as e:
        print(f"⚠️ Error at offset {offset}: {e}")
        break

    print(
        f"\n✨ Full dictionary scraping complete! Total unique entries saved:"
        f" {total_new_saved}"
    )

    # Check final count in SQLite database
    conn = sqlite3.connect(DB_FILE)
    total_count = conn.execute("SELECT COUNT(*) FROM dictionary").fetchone()[0]
    conn.close()
    print(f"🗄️ Total unique entries now in 'lotto.db': {total_count}")


if __name__ == "__main__":
    scrape_all_via_load_more()
```

```python
import json
import sqlite3
from fastapi import FastAPI, HTTPException
import re

app = FastAPI(
    title="West Malaysia 4D History API",
    version="1.0",
    description=(
        "Query historical 4D draw results and search for specific winning"
        " numbers."
    ),
)

DB_FILE = "lotto.db"


def get_db():
  conn = sqlite3.connect(DB_FILE)
  conn.row_factory = sqlite3.Row
  return conn


@app.get("/draws", summary="Get Historical Draws")
def get_draws(company: str = None, limit: int = 10):
  """Fetch records with optional filtering by operator company (Magnum 4D, Damacai, Sports Toto)."""
  conn = get_db()
  cursor = conn.cursor()

  if company:
    cursor.execute(
        "SELECT * FROM draws WHERE company = ? ORDER BY draw_date DESC LIMIT"
        " ?",
        (company, limit),
    )
  else:
    cursor.execute("SELECT * FROM draws ORDER BY draw_date DESC LIMIT ?", (limit,))

  rows = cursor.fetchall()
  conn.close()

  results = []
  for row in rows:
    results.append({
        "company": row["company"],
        "date": row["draw_date"],
        "first": row["first_prize"],
        "second": row["second_prize"],
        "third": row["third_prize"],
        "special": json.loads(row["special_prizes"]),
        "consolation": json.loads(row["consolation_prizes"]),
    })
  return results


@app.get("/search/{number}", summary="Search 4D Number History")
```

```python
def search_number(number: str):
  """Check all historical draws to see when a specific 4-digit number appeared."""
  if not re.fullmatch(r"\d{4}", number):
    raise HTTPException(
        status_code=400, detail="Invalid format. Must be strictly 4 digits."
    )

  conn = get_db()
  cursor = conn.cursor()
  cursor.execute(
      """
        SELECT * FROM draws
        WHERE first_prize = ? OR second_prize = ? OR third_prize = ?
        OR special_prizes LIKE ? OR consolation_prizes LIKE ?
        ORDER BY draw_date DESC
    """,
      (number, number, number, f"%{number}%", f"%{number}%"),
  )
  rows = cursor.fetchall()
  conn.close()

  if not rows:
    raise HTTPException(
        status_code=404, detail=f"Number {number} not found in records."
    )

  matches = [{"company": r["company"], "date": r["draw_date"]} for r in rows]
  return {"number": number, "total_hits": len(matches), "matches": matches}
```

```python
from datetime import datetime, timedelta
import json
import os
import re
import sqlite3
import time
from bs4 import BeautifulSoup
import requests

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_FILE = os.path.join(BASE_DIR, "lotto.db")
OUTPUT_FILE = os.path.join(BASE_DIR, "west_malaysia_10years_4d.json")


def get_latest_draw_date():
  """Finds the most recent draw date recorded in the SQLite database."""
  if not os.path.exists(DB_FILE):
    return datetime(2016, 1, 1)

  try:
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("SELECT MAX(draw_date) FROM draws")
    result = cursor.fetchone()
    conn.close()

    if result and result[0]:
      return datetime.strptime(result[0], "%Y-%m-%d")
  except Exception as e:
    print(f"⚠️ Error querying latest date: {e}")
  return datetime(2016, 1, 1)


def save_new_record(record):
  """Stores new updates into database and JSON backup."""
  try:
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute(
        """
            INSERT OR IGNORE INTO draws
            (company, draw_date, first_prize, second_prize, third_prize, special_prizes, consolation_prizes)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        """,
        (
            record["company"],
            record["date"],
            record["prizes"]["first"],
            record["prizes"]["second"],
            record["prizes"]["third"],
            json.dumps(record["prizes"]["special"]),
            json.dumps(record["prizes"]["consolation"]),
        ),
    )
    conn.commit()
    conn.close()
```

```
    if os.path.exists(OUTPUT_FILE):
      with open(OUTPUT_FILE, "r", encoding="utf-8") as f:
        data = json.load(f)
    else:
      data = []

    data.append(record)
    with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
      json.dump(data, f, indent=4, ensure_ascii=False)

    print(
        f"✅ [New Draw Recorded] {record['company']} | Date:"
        f" {record['date']} | 1st: {record['prizes']['first']}"
    )
  except Exception as e:
    print(f"❌ [Error] {e}")


def check_and_update():
  latest_db_date = get_latest_draw_date()
  start_dt = latest_db_date + timedelta(days=1)
  end_dt = datetime.now()

  if start_dt > end_dt:
    print("✨ Database is already fully up to date.")
    return

  missing_dates = []
  curr = start_dt
  while curr <= end_dt:
    if curr.weekday() in [2, 5, 6]:  # Wed, Sat, Sun
      missing_dates.append(curr)
    curr += timedelta(days=1)

  if not missing_dates:
    print("✨ No new draw dates to check.")
    return

  print(
      f"🚀 Checking {len(missing_dates)} potential new draw date(s)..."
  )
  headers = {
      "User-Agent": (
          "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,"
          " like Gecko) Chrome/122.0.0.0 Safari/537.36"
      )
  }

  for d in missing_dates:
    date_str = d.strftime("%Y-%m-%d")
    url = f"https://www.4dmoon.com/past-results/{date_str}"
    try:
      time.sleep(0.3)
      response = requests.get(url, headers=headers, timeout=15)
      if response.status_code == 200:
        soup = BeautifulSoup(response.text, "html.parser")
```

```
        for container in soup.find_all(["div", "section", "table"]):
          text_content = container.get_text()
          if any(op in text_content for op in ["Magnum 4D", "Damacai", "SportsToto"]):
            company_name = (
                "Damacai"
                if "Damacai" in text_content
                else ("Magnum 4D" if "Magnum" in text_content else "Sports Toto")
            )
            cells = container.find_all(["td", "div", "span", "p"])
            numbers = [
                c.get_text().strip()
                for c in cells
                if re.fullmatch(r"\d{4}", c.get_text().strip())
            ]
            # Deduplicate while preserving order
            numbers = list(dict.fromkeys(numbers))

            if len(numbers) >= 3:
              record = {
                  "company": company_name,
                  "date": date_str,
                  "prizes": {
                      "first": numbers[0],
                      "second": numbers[1],
                      "third": numbers[2],
                      "special": numbers[3:13]
                      if len(numbers) >= 13
                      else numbers[3:],
                      "consolation": numbers[13:23]
                      if len(numbers) >= 23
                      else [],
                  },
              }
              save_new_record(record)
    except Exception as e:
      print(f"⚠️ Error on {date_str}: {e}")


if __name__ == "__main__":
  check_and_update()
```

```python
import os
import re
import sqlite3
import ollama

# ==========================================
# CONFIGURATION & SETUP
# ==========================================
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_FILE = os.path.join(BASE_DIR, "lotto.db")
LOCAL_MODEL = "llama3.2"

print(f"[*] Initializing Local AI Analyzer...")
print(f"[*] Database Path: {DB_FILE}")


def get_db_connection():
    """Establishes a connection to the SQLite database."""
    if not os.path.exists(DB_FILE):
        print(f"[!] Warning: Database file not found at {DB_FILE}.")
        return None
    return sqlite3.connect(DB_FILE)


def search_dictionary_and_draws(query_text):
    """Performs precise SQL lookups on dictionary terms and draw results

    based on tokens found in the user prompt.
    """
    conn = get_db_connection()
    if not conn:
        return {"dictionary": [], "draws": []}

    cursor = conn.cursor()
    tokens = [t.lower() for t in re.findall(r"\w+", query_text) if len(t) > 2]
    dict_results = []
    draw_results = []

    # 1. Search dictionary table for semantic matches
    for token in tokens:
        cursor.execute(
            "SELECT term, number FROM dictionary WHERE LOWER(term) LIKE ?",
            (f"%{token}%",),
        )
        for row in cursor.fetchall():
            if row not in dict_results:
                dict_results.append(row)

    # 2. Check if a specific date was requested (YYYY-MM-DD)
    date_match = re.search(r"\d{4}-\d{2}-\d{2}", query_text)
    if date_match:
        target_date = date_match.group(0)
        cursor.execute(
            "SELECT company, draw_date, first_prize, second_prize, third_prize FROM draws
WHERE draw_date = ?",
            (target_date,),
        )
```

```
        draw_results = cursor.fetchall()
    else:
        # Fetch recent draws as context
        cursor.execute(
            "SELECT company, draw_date, first_prize FROM draws ORDER BY draw_date DESC LIMIT 3"
        )
        draw_results = cursor.fetchall()

    conn.close()
    return {"dictionary": dict_results, "draws": draw_results}


def query_local_ai_with_context(user_query):
    """Gathers local DB context and queries the local Ollama AI model."""
    print(f"\nUser Query: '{user_query}'")
    print("-" * 60)

    # Retrieve factual data from SQLite database first
    db_data = search_dictionary_and_draws(user_query)

    # Strict context prompt to prevent hallucinations
    context_prompt = f"""
[DATA CONTEXT]
- Dictionary Match Results (Term & Numbers): {db_data['dictionary']}
- Draw Records Results (Company, Date, Prizes): {db_data['draws']}

[USER REQUEST]
{user_query}

[INSTRUCTIONS]
1. Answer the user request strictly using the provided [DATA CONTEXT] above.
2. If the [DATA CONTEXT] is empty or does not contain the answer, explicitly state: "No matching records found in the local database."
3. Do not invent, assume, or hallucinate lottery numbers or dictionary mappings that are not present in the data context.
4. Keep the response concise, factual, and direct (under 75 words).
"""

    try:
        response = ollama.chat(
            model=LOCAL_MODEL,
            messages=[
                {
                    "role": "system",
                    "content": (
                        "You are a precise data-retrieval assistant. You never "
                        "hallucinate facts and rely solely on the provided database context."
                    ),
                },
                {"role": "user", "content": context_prompt},
            ],
            options={"temperature": 0.1},
        )

        print("[Local AI Response]:")
```

```
        print(response["message"]["content"].strip())

    except Exception as e:
        print(
            f"[!] Error communicating with local Ollama service. Ensure `ollama serve` is
running. Details: {e}"
        )


# ========================================
# DUAL MODE INTERFACE
# ========================================
def run_test_mode():
    """Runs a pre-defined batch of standard test questions."""
    print("\n=== RUNNING TEST MODE (Pre-defined Questions) ===")
    test_queries = [
        "Can you check the lottery result for 2016-04-02?",
        "I dreamed about a big snake last night, what numbers match that?",
        "Show me the latest draw results.",
    ]

    for i, query in enumerate(test_queries, 1):
        print(f"\n[Test Case {i}]")
        query_local_ai_with_context(query)
        print("=" * 60)


def run_client_mode():
    """Allows users to input their own questions interactively."""
    print("\n=== RUNNING CLIENT MODE (Interactive) ===")
    print("Type your questions below. Type 'exit' or 'quit' to stop.\n")

    while True:
        try:
            user_input = input("Client Query > ").strip()
            if user_input.lower() in ["exit", "quit"]:
                print("Exiting client mode. Goodbye!")
                break
            if not user_input:
                continue

            query_local_ai_with_context(user_input)
            print("=" * 60)
        except KeyboardInterrupt:
            print("\nExiting client mode.")
            break


if __name__ == "__main__":
    print("\nSelect Mode:")
    print("1. Test Mode (Run preset evaluation questions)")
    print("2. Client Mode (Write your own questions interactively)")

    choice = input("Enter choice (1 or 2): ").strip()

    if choice == "1":
        run_test_mode()
```

```
166     elif choice == "2":
167         run_client_mode()
168     else:
169         print("Invalid choice. Defaulting to Test Mode.")
170         run_test_mode()
```